

Casino Royale 3D

Relatorio de Projeto ICG 2025/2026

Guilherme Tavares 119867

27 de maio de 2026

1 Resumo

O *Casino Royale 3D* é uma aplicação WebGL em Three.js que coloca o jogador no interior de uma sala de casino explorável em primeira pessoa, com três jogos completos (Blackjack, Poker de mesa e Roleta Europeia), seis máquinas de slots, multibanco, bar e personagens. O projeto demonstra a pipeline gráfica moderna no browser, transformações hierárquicas via grafo de cena, materiais PBR com texturas geradas em **Canvas**, iluminação com sombras suaves, interação em tempo real com teclado, rato e toque, e um conjunto de animações baseadas em *tweens* e *easings* que materializam mecânicas de jogo concretas.

Este relatório descreve a arquitetura técnica, os modelos e cenas, a estratégia de iluminação, e formaliza matematicamente as principais mecânicas: trajetória da bola da roleta, animação dos reels das slots, embaralhamento e distribuição de cartas, avaliação de mãos de Blackjack/Poker, transições de câmara e decaimento de sobriedade.

2 Visão do projeto

Genero. Simulador de casino em primeira pessoa, com três jogos de mesa funcionais, slots, bar e multibanco.

Objetivo do jogador. Gerir um saldo único de fichas entre as várias mesas, com a possibilidade de reforço no multibanco, e interagir com o ambiente (bar, slots) como atividades secundárias.

Componentes principais.

- Sala interior modelada proceduralmente: piso, paredes, pilares, mesas, candelabro, ventiladores, quadros, plantas, bar com balcão e estantes de garrafas, multibanco, e seis máquinas de slots.
- Três jogos completos: Blackjack, Poker de mesa (com três bots e fases de aposta) e Roleta Europeia.
- Sistemas complementares: bar com bebida em primeira pessoa, decadência de sobriedade, blackout, multibanco com saldo partilhado.
- Câmera contextual: primeira pessoa em exploração, vista superior em mesas, com transições suaves *slerp/lerp*.
- Suporte mobile/tablet: joystick virtual, botão de interação, sprint, sair e olhar por arrasto.

3 Interação com o jogador

3.1 Inputs

Em desktop são usados teclado e rato com `PointerLock`. Em dispositivos com `pointer`: `coarse` ou `maxTouchPoints > 0` são criados controlos por toque.

Input	Acao
W/A/S/D	Movimento horizontal
Shift	Correr
Rato (apos clique)	Olhar (pointer lock)
E	Interagir com a zona ativa
F	Beber (frente ao bar)
Esc	Sair de mesa / libertar pointer
Joystick virtual	Movimento (mobile)
Area direita do canvas	Olhar (mobile)
Botao Interagir	Interacao (mobile)

3.2 Maquina de estados

O jogo opera numa maquina de estados deterministica:

```
splash -> exploring -> blackjack
      -> poker
      -> roulette
      -> atm
      -> bar
      -> blackout (consumo excessivo)
```

A transicao `exploring -> X` salva a pose da camara, desativa o controlador FPS e inicia uma animacao. A transicao `X -> exploring` restaura a pose e reativa o FPS. Estados de jogo (`blackjack/poker/roulette`) so podem ser activados a partir de `exploring` para prevenir re-entradas durante transicoes.

4 Modulos e ferramentas

- **Three.js** (r0.183) – pipeline WebGL, classes de geometria, materiais, luzes e camaras.
- **Three.js addons** – `RoomEnvironment`, `PMREMGGenerator`, `RectAreaLightUniformsLib`.
- **Vite** (v7) – servidor de desenvolvimento e build de producao com publicacao em `dist/`.
- **Terser** – minificacao da build final.
- **GitHub Actions** – workflow de deploy automatico em GitHub Pages.

Nao sao usados motores de fisica externos: as colisoes sao AABB simples e as animacoes sao construidas por *tweens* promissorios e funcoes de easing implementadas no proprio projeto.

5 Modelos e grafo de cena

5.1 Composicao hierarquica

Todos os objetos sao construidos a partir de primitivas (`BoxGeometry`, `CylinderGeometry`, `TorusGeometry`, `SphereGeometry`, `ConeGeometry`, `CircleGeometry`, `PlaneGeometry`) e agrupados em `THREE.Group`, de modo a permitir transformacoes locais e propagacao hierarquica. Por exemplo, uma cadeira e um `Group` com base, encosto e quatro pernas; uma mesa de poker e um `Group` com tampo, feltro, aro, coluna central, seis cadeiras e suportes de copos.

5.2 Resumo do grafo

```
scene
+- camera (PerspectiveCamera + mug attached no bar)
+- Luzes
    AmbientLight, HemisphereLight, DirectionalLight (shadow)
```

```

    SpotLights nas mesas, RectAreaLight no bar
    PointLights no candelabro e apliques
+- Sala (CasinoWorld)
    +- chao + carpete (CircleGeometry + texturas)
    +- paredes, pilares, ventiladores
    +- mesas de Blackjack/Poker/Roleta (Group)
    +- bar (Group: balcao, prateleiras, garrafas)
    +- ATM (Group)
    +- slot machines (Group x6, cada com reelGroup x3)
    +- candelabro, quadros, plantas
    +- personagens (dealer, croupier, bots, bartender)

```

5.3 Modelacao procedural

A modelacao integralmente procedural permite uniformidade visual e facilidade de variacao (cores, dimensoes, repeticoes). Por exemplo, um candelabro central e construido por iteracao angular:

$$\begin{aligned} x_i &= R_{\text{ring}} \cos\left(\frac{2\pi i}{N}\right), \\ z_i &= R_{\text{ring}} \sin\left(\frac{2\pi i}{N}\right), \end{aligned} \quad i = 0, \dots, N-1, \quad (1)$$

sendo posicionados N bracos numa coroa de raio R_{ring} . A roleta usa a mesma formulacao para distribuir as 37 casas:

$$\theta_i = \theta_0 + \frac{2\pi \pi(i)}{37}, \quad (2)$$

onde $\pi(\cdot)$ e a permutacao da ordem real da roleta europeia (WHEEL_ORDER).

6 Iluminacao, materiais e texturas

6.1 Iluminacao

A sala foi iluminada como um casino noturno, equilibrando legibilidade do espaco e ambiente cinematografico:

- **AmbientLight** e **HemisphereLight** de baixa intensidade – garantem que zonas em sombra nao colapsam a preto.
- Uma unica **DirectionalLight** com mapa de sombras (**PCFSoftShadowMap**) define a sombra principal.
- **SpotLights** dedicados sobre Blackjack, Poker e Roleta, com abajures visiveis e difusores emissivos.
- **RectAreaLight** sobre o bar para realcar o marmore e o vidro das garrafas.
- **PointLights** no candelabro central e em apliques de parede (sem sombra, para ambiente).

A estrategia e ter *uma* luz com sombra real (a direccional) e usar emissive + point lights sem sombra para os detalhes decorativos. Isto preserva FPS estavel mesmo com muitos elementos luminosos visiveis.

6.2 Tone mapping e cor

O renderer usa **ACESFilmicToneMapping** para preservar *highlights* de fontes emissivas e **outputColorSpace = SRGBColorSpace** para producao correta no browser. Em dispositivos sem **lowPower**, e gerado um *environment map* a partir de **RoomEnvironment** via **PMREMGGenerator**, dando reflexoes neutras plausiveis em cromados, marmore, ouro e vidro – complementando rasterizacao com *image-based lighting*.

6.3 Materiais

- `MeshStandardMaterial` – maioria das superficies (madeira, carpete, paredes, fichas, cartas).
- `MeshPhysicalMaterial` – bar (com `clearcoat` para o marmore), maquinas de slots (corpo em laca).
- Materiais emissivos – bulbos do candelabro, neon, sinais das slots, feltro retroiluminado.

6.4 Texturas procedurais

As texturas grandes (carpete, madeira do bar, marmore, feltro, paredes, teto) sao geradas em runtime em `ProceduralTextures.js` por desenho num `HTMLCanvasElement` 2D e depois envolvidas em `CanvasTexture`. Cada conjunto inclui:

- `map` – cor de superficie (*albedo*).
- `normalMap` – relevo aparente (perturbacao da normal por gradiente Sobel local).
- `roughnessMap` – variacao espacial do brilho.

A repeticao UV usa `RepeatWrapping` e e parametrizada por superficie (*e.g.*, a carpete repete 8x para nao se notar *tile*, enquanto o feltro das mesas repete 1x).

7 Animacoes e mecanicas

7.1 Tween engine

A engine de animacao (`AnimationManager`) e um pequeno motor de tweens baseado em Promises:

```
animate(target, prop, endVal, duration, easeFn) -> Promise
```

Os tweens activos sao processados a cada frame:

$$v(t) = v_0 + (v_1 - v_0) \cdot E(t), \quad t = \frac{\text{now} - t_0}{\Delta T}, \quad (3)$$

onde $E(t)$ e uma das funcoes de easing usadas:

$$E_{\text{cubic}_{\text{io}}}(t) = \begin{cases} 4t^3, & t < 0.5 \\ 1 - \frac{(-2t+2)^3}{2}, & t \geq 0.5 \end{cases} \quad (4)$$

$$E_{\text{cubic}_{\text{out}}}(t) = 1 - (1 - t)^3 \quad (5)$$

$$E_{\text{back}_{\text{out}}}(t) = 1 + c_3(t - 1)^3 + c_1(t - 1)^2, \quad c_1 = 1.70158, \quad c_3 = c_1 + 1 \quad (6)$$

$$E_{\text{elastic}_{\text{out}}}(t) = 2^{-10t} \sin((10t - 0.75)\frac{2\pi}{3}) + 1 \quad (7)$$

A animacao do candelabro/ventiladores incrementa θ_y com base no **delta time** para independencia do FPS:

$$\theta_y \leftarrow \theta_y + \omega \cdot \Delta t. \quad (8)$$

7.2 Idle de personagens

Em `AnimationManager.update`, cada personagem (registada via `registerCharacter`, com $\phi = \text{breathePhase}$ e $y_0 = \text{baseY}$ definidos na construcao em `SceneElements.js`) e animada a cada frame:

$$y(t) = y_0 + A_y \sin(1.3t + \phi), \quad (9)$$

$$\theta_{\text{cabeca},z}(t) = A_z \sin(0.7t + \phi), \quad (10)$$

$$\theta_{\text{cabeca},x}(t) = A_x \sin(0.5t + 2\phi). \quad (11)$$

As reacoes (*nod*, *shake*, *surprise*, *celebrate*, *think*, *fold*, *check*, *call*, *raise*), em `AnimationManager.characterReact` sao sequencias de tweens em rotacoes da cabeca e/ou bracos. As reacoes de confirmacao como *nod*, *think* e *fold* foram mantidas na cabeca para evitar que o corpo inteiro se desloque artificialmente; *check*, *call* e *raise* usam o braco para comunicar a acao da aposta; *celebrate* e *surprise* podem mover tambem o corpo por serem reacoes fisicas mais fortes. Sao disparadas pelos controladores conforme o estado de jogo: o *dealer* de Blackjack faz *nod/shake/surprise*, o *croupier* da Roleta faz *think/nod/shake*, e os *bots* de Poker fazem *think/fold/check/call/raise/surprise/celebrate*.

8 Modelos matematicos e logica de jogo

8.1 Embaralhamento Fisher–Yates

O baralho de 52 cartas e embaralhado pelo algoritmo de Fisher–Yates, com complexidade $\mathcal{O}(n)$ e distribuicao uniforme sobre as $n!$ permutacoes possiveis:

```
for i = n - 1 down to 1 :
    j ← ⌊U · (i + 1)⌋,    U ∼ Unif[0, 1)
    swap(ai, aj)
```

(12)

Implementacao em `src/cards/Deck.js`:

```
for (let i = this.cards.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [this.cards[i], this.cards[j]] = [this.cards[j], this.cards[i]];
}
```

8.2 Distribuicao e animacao de cartas

Implementada em `AnimationManager.dealCard`, cada carta atravessa tres fases. Sejam $\mathbf{p}_s$ a posicao inicial (perto do baralho) e $\mathbf{p}_f$ a posicao final (a frente do jogador). Define-se um ponto intermedio (no codigo, `midX = (mesh.x + targetX) * 0.55`, identico em z):

$$\mathbf{p}_m = 0.55\mathbf{p}_s + 0.55\mathbf{p}_f, \quad (13)$$

e a carta:

1. **Slide**: $\mathbf{p}(t) = \text{lerp}(\mathbf{p}_s, \mathbf{p}_m, E_{\text{cubic}_{\text{io}}}(t))$ durante 160 ms, com pequena rotacao em y e escala $0.85 \rightarrow 0.92$.
2. **Land**: $\mathbf{p}(t) = \text{lerp}(\mathbf{p}_m, \mathbf{p}_f, E_{\text{cubic}_{\text{out}}}(t))$ durante 200 ms, com escala $0.92 \rightarrow 1$ e rotacao em y a 0.
3. **Flip**: rotacao $\theta_z : \pi \rightarrow 0$ com $E_{\text{back}_{\text{out}}}$ durante 280 ms, sobreposta com uma elevacao em y de +0.06 e retorno.

8.3 Blackjack: valor de mao e pagamentos

A logica esta em `src/games/BlackjackGame.js` (`getHandValue` e `getPayout`). Cada carta tem valor base $v(c) \in \{2, 3, \dots, 10, 10, 10, 10\}$ para numericas e figuras, e $v(A) = 11$ inicialmente para o As. O valor *naive* de uma mao $H = \{c_1, \dots, c_n\}$:

$$S = \sum_{c \in H} v(c), \quad (14)$$

e ajustado para evitar *bust* com Ases macios: `getHandValue` subtrai 10 (As de 11 para 1) repetidamente enquanto a mao excede 21 e ainda ha Ases por reduzir. O numero de reducoes e

$$S' = S - 10k, \quad k = \min(a, \max(0, \lceil \frac{S-21}{10} \rceil)), \quad (15)$$

onde a e o numero de Ases. O tecto $\lceil \cdot \rceil$ (e nao $\lfloor \cdot \rfloor + 1$) garante $k = 0$ quando $S \leq 21$, preservando um *soft* 21. O dealer pede cartas enquanto $S'_D < 17$. O pagamento sobre uma aposta b :

$$P(b, S'_J, S'_D) = b \cdot \begin{cases} 2.5, & \text{blackjack natural } (n = 2 \wedge S'_J = 21) \\ 2, & S'_J \leq 21 \wedge (S'_D > 21 \vee S'_J > S'_D) \\ 1, & S'_J = S'_D \wedge S'_J \leq 21 \\ 0, & \text{caso contrario} \end{cases} \quad (16)$$

8.4 Poker de mesa

Conjunto avaliado. Cada jogador j tem cartas privadas H_j com $|H_j| = 2$ e a mesa tem C_{mesa} com $|C_{\text{mesa}}| \leq 5$. A mao avaliada e:

$$C_j = H_j \cup C_{\text{mesa}}, \quad 2 \leq |C_j| \leq 7. \quad (17)$$

Avaliacao. O avaliador (`evaluateHand` em `src/games/PokerHand.js`) recebe $|C_j| \geq 5$, percorre todas as combinacoes de 5 cartas em C_j e devolve a categoria R com maior pontuacao, com desempate por *high cards*:

$$R^*(C_j) = \max_{S \subseteq C_j, |S|=5} R(S), \quad |\{S\}| = \binom{|C_j|}{5} \leq 21. \quad (18)$$

A escala de categorias e:

$$R \in \{0, 1, \dots, 9\} = \{\text{High Card}, \text{Pair}, \dots, \text{Royal Flush}\}. \quad (19)$$

Vencedor. Entre os jogadores activos J :

$$j^* = \arg \max_{j \in J} (R^*(C_j), \text{HC}(C_j)), \quad (20)$$

com a ordenacao lexicografica em (R^*, HC) , onde HC e a lista ordenada de *kick*ers.

Heuristica dos bots. A decisao esta em `PokerGame.botDecision` (`src/games/PokerGame.js`). *Pos-flop* (com cartas comunitarias), a forca base e a categoria avaliada:

$$F = \text{clip}\left(\frac{R^*(C_j)+1}{10} + \epsilon, 0, 1\right), \quad \epsilon \sim \text{Unif}[0, 0.15]; \quad (21)$$

pre-flop usa antes uma heuristica das duas cartas privadas $((v_1 + v_2)/28$ com bonus por par, *suited* e *connectors*). A forca efetiva soma ainda um bonus de posicao e uma agressividade propria de cada bot:

$$F_{\text{ef}} = \min(1, F + \beta_{\text{pos}} + \alpha_{\text{bot}}), \quad (22)$$

e o bot decide entre *fold*, *check*, *call*, *raise* ou *all-in* comparando F_{ef} (e uma probabilidade de *bluff*) com limiares e com a relacao $b_{\text{call}}/\text{pot}$. O pote P evolui como:

$$P_{t+1} = P_t + b_{j,t}. \quad (23)$$

8.5 Roleta: trajetoria da bola

A trajetoria e gerada em `RouletteController._animateSpinToPocket`, num *loop* de `requestAnimationFrame` de duracao $T = 12.5$ s. A roleta combina duas rotacoes (roda e bola) em coordenadas polares centradas no eixo y . A roda tem aceleracao monotona com easing:

$$\theta_{\text{wheel}}(t) = \theta_{w,0} + \Delta\theta_w \cdot (1 - (1 - t)^{2.4}), \quad (24)$$

sendo $\Delta\theta_w = 2\pi \cdot (2.8 + \xi)$ com $\xi \sim \text{Unif}[0, 0.35]$.

A bola percorre tres fases em $t \in [0, 1]$ de duracao total $T = 12.5$ s:

Fase 1 – Ganho ($t < 0.24$). Na pista exterior, raio $r = r_{\text{out}}$, altura $y = y_{\text{out}}$, com aceleracao quadratica:

$$u = \frac{t}{0.24}, \quad \theta_{\text{ball}}(t) = \theta_{b,0} + 0.18 \cdot (\theta_{b,f} - \theta_{b,0}) \cdot u^2. \quad (25)$$

Fase 2 – Largada ($0.24 \leq t < 0.48$). Transicao da pista exterior para a interior:

$$u = \frac{t-0.24}{0.24}, \quad (26)$$

$$r(t) = r_{\text{out}} + (r_{\text{in}} - r_{\text{out}}) \cdot E_{\text{cubic}_{\text{io}}}(u), \quad (27)$$

$$y(t) = y_{\text{out}} + (y_{\text{in}} - y_{\text{out}}) \cdot E_{\text{cubic}_{\text{io}}}(u), \quad (28)$$

$$\theta_{\text{ball}}(t) = \theta_A + (\theta_B - \theta_A) (1 - (1 - u)^{1.45}), \quad (29)$$

com $\theta_A = \theta_{b,0} + 0.18\Delta\theta_b$ e $\theta_B = \theta_{b,0} + 0.50\Delta\theta_b$, onde $\Delta\theta_b = \theta_{b,f} - \theta_{b,0}$.

Fase 3 – Casas e pouso ($0.48 \leq t < 0.985$). A bola atravessa as casas e desacelera continuamente ate ao bolso final:

$$u = \frac{t-0.48}{0.505}, \quad (30)$$

$$E(u) = 1 - (1 - u)^{2.8}, \quad (31)$$

$$\theta_{\text{ball}}(t) = \theta_B + (\theta_{b,f} - \theta_B) E(u) + A_r \sin(34\pi u + \varphi) (1 - u), \quad (32)$$

$$r(t) = r_{\text{in}} + (r_{\text{pocket}} - r_{\text{in}}) E(u) + A_{rr} \sin(18\pi u) (1 - u), \quad (33)$$

$$y(t) = y_{\text{in}} + (y_{\text{pocket}} - y_{\text{in}}) E(u) + A_{yy} |\sin(20\pi u)| (1 - u), \quad (34)$$

com $A_r = 0.035$, $A_{rr} = 0.008$, $A_{yy} = 0.018$ e φ uma fase aleatoria. O termo sinusoidal $\sin(34\pi u + \varphi)(1 - u)$ modela o *rattle* amortecido contra os separadores das casas. Esta fase substitui um anterior *low-energy crawl* adicional que percorria meia volta extra a baixa velocidade e nao parecia natural.

Fase 4 – Vibracao residual ($t \geq 0.985$). Pequena vibracao amortecida no bolso vencedor:

$$\theta(t) = \theta_{b,f} + 0.004 \sin(2.5\pi u) e^{-7u}, \quad u = \frac{t-0.985}{0.015}. \quad (35)$$

Numero de voltas. A bola gira no sentido inverso da roda durante $N_{\text{laps}} \in [11, 14]$ voltas, e o angulo final do bolso e:

$$\theta_{b,f} = \theta_w(T) + \theta_{\text{pocket}} - 2\pi N_{\text{laps}}, \quad \theta_{\text{pocket}} = \theta_0 + \frac{2\pi \pi(n^*)}{37}, \quad (36)$$

onde $\pi(n^*)$ e o indice do numero vencedor n^* na ordem real da roleta europeia.

Pagamentos. A logica de aposta esta em `src/games/RouletteGame.js` (`spin`), com o pagamento $P = b(m + 1)$ para vitoria e 0 caso contrario, onde m e o multiplicador. Numa aposta b :

$$P = \begin{cases} 36b, & \text{numero direto acertado, ou verde (zero) acertado} \quad (m = 35) \\ 2b, & \text{aposta em vermelho/preto acertada} \quad (m = 1) \\ 0, & \text{caso contrario} \end{cases} \quad (37)$$

Note-se que o verde, embora seja uma aposta de cor, paga como o numero (35:1) por corresponder a uma unica casa.

8.6 Slots: animacao dos reels e regras

A logica e a animacao estao em `src/main.js` (`playSlotMachine` e `_animateSlotReel`). Cada uma das seis maquinas tem tres *reel groups*, cada um com uma face de fundo e um simbolo (cor codificada). Quando o jogador interage, o resultado e amostrado uniformemente:

$$r_i \sim \text{Unif}\{0, 1, 2\}, \quad i \in \{1, 2, 3\}, \quad (38)$$

para `[vermelho, verde, amarelo]`.

Cada reel anima a sua rotacao em torno do eixo x local com duracao escalonada $T_i \in \{1.10, 1.45, 1.75\}$ s:

$$\theta_{x,i}(t) = 16\pi \cdot E_{\text{cubic}_{\text{out}}}\left(\frac{t}{T_i}\right) = 16\pi \cdot \left(1 - \left(1 - \frac{t}{T_i}\right)^3\right), \quad (39)$$

o que equivale a 8 voltas completas com aceleracao no inicio (ou seja, com desaceleracao em *ease-out*). Em paralelo, a cor do simbolo cicla a cada $\Delta t = 70$ ms:

$$c_i(t) = \mathbf{C}[\lfloor t/\Delta t \rfloor \bmod 3], \quad t < 0.88 T_i, \quad (40)$$

com $\mathbf{C}$ a paleta `[\#D32F2F, \#2E7D32, \#FBC02D]`. No instante $t = T_i$ o reel assenta em $\theta_{x,i} = 0$ e a cor fica $\mathbf{C}[r_i]$. O pagamento sobre uma aposta $b = 10$:

$$P = \begin{cases} 250, & r_1 = r_2 = r_3 = 2 \quad (\text{jackpot amarelo}) \\ 100, & r_1 = r_2 = r_3 \neq 2 \\ 0, & \text{caso contrario} \end{cases} \quad (41)$$

A regra simplifica deliberadamente a payout table de maquinas reais para que o jogador veja claramente que so 3 iguais ganham (sem premios parciais por 2 iguais).

8.7 Transicoes de camara

A transicao entre exploracao e mesa interpola posicao e orientacao em paralelo. Sejam $\mathbf{p}_0, \mathbf{p}_1$ as posicoes e $\mathbf{q}_0, \mathbf{q}_1$ os quaternioes:

$$\mathbf{p}(t) = \text{lerp}(\mathbf{p}_0, \mathbf{p}_1, E_{\text{cubic}_{\text{io}}}(t)), \quad (42)$$

$$\mathbf{q}(t) = \text{slerp}(\mathbf{q}_0, \mathbf{q}_1, E_{\text{cubic}_{\text{io}}}(t)), \quad (43)$$

com `slerp` definido em `THREE.Quaternion`:

$$\text{slerp}(\mathbf{q}_0, \mathbf{q}_1, t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} \mathbf{q}_0 + \frac{\sin(t\Omega)}{\sin \Omega} \mathbf{q}_1, \quad \cos \Omega = \mathbf{q}_0 \cdot \mathbf{q}_1. \quad (44)$$

A duracao usada e 700 ms para entrada/saida de mesas (Blackjack, Poker, Roleta) e 500 ms para sistemas modais (ATM, Bar, retorno).

8.8 Interacao por proximidade

Gerida por `InteractionSystem` (`src/scene/InteractionSystem.js`), a cada frame em estado `exploring`, para cada zona de interacao z com posicao $\mathbf{p}_z$ e raio ρ_z , calcula-se a distancia planar (`Math.hypot`):

$$d(\mathbf{p}_{\text{cam}}, z) = \sqrt{(x - z_x)^2 + (z - z_z)^2}, \quad (45)$$

e a zona ativa z^* e:

$$z^* = \arg \min_{z : d(\mathbf{p}_{\text{cam}}, z) < \rho_z} d(\mathbf{p}_{\text{cam}}, z), \quad (46)$$

ou $\emptyset$ se nenhuma zona obedece a $d < \rho_z$. Quando z^* muda, mostra-se ou esconde-se o prompt correspondente.

8.9 Decaimento de sobriedade

Implementada em `src/main.js` (variaveis `drinksCount` e `soberAccumulator`), cada bebida incrementa $D \leftarrow D + 1$. O nivel de embriaguez visual e:

$$L = \min\left(1, \frac{D}{20}\right), \quad (47)$$

e e usado para distorcer a camara (oscilacoes em θ_x e θ_z) e adicionar borrado de imagem. Em paralelo, um acumulador σ cresce com o tempo real ($\sigma \leftarrow \sigma + \Delta t$) e converte-se em redução de bebidas a cada 60 segundos:

$$D \leftarrow \max(0, D - \lfloor \frac{\sigma}{60} \rfloor), \quad \sigma \leftarrow \sigma \bmod 60. \quad (48)$$

Quando $D = 20$, e despoletado o *blackout*: queda animada da camara, *fade* para preto e `location.reload()`.

9 Arquitetura tecnica

- **Camara 3D** – `PerspectiveCamera` unica, em primeira pessoa, com transicoes *slerp* para vistas superiores. O *far* adapta-se ao `lowPower`.
- **Input** – separacao entre `FPSControls` (teclado + pointer lock) e `TouchControls` (joystick virtual, botoes, look por arrasto). O `main.js` sincroniza o estado de movimento por frame, no padrao $\mathbf{v}(t) = f(\text{input}, \Delta t)$.
- **Fisicas** – nao ha motor externo. As colisoes usam caixas AABB no plano xz (`FPSControls._checkCollision`). O movimento e resolvido eixo a eixo: a partir de (x_0, z_0) , testam-se independentemente as posicoes candidatas (x', z_0) e (x_0, z') , aceitando cada componente apenas se nao houver sobreposicao com nenhuma caixa – o que produz *deslize* ao longo das paredes em vez de bloqueio total:

$$x \leftarrow \begin{cases} x', & \neg \text{ovl}(x', z_0) \\ x_0, & \text{c.c.} \end{cases} \quad z \leftarrow \begin{cases} z', & \neg \text{ovl}(x_0, z') \\ z_0, & \text{c.c.} \end{cases}$$

onde $\text{ovl}(x, z)$ e verdadeiro sse, para alguma caixa de limites $[x_{\min}, x_{\max}] \times [z_{\min}, z_{\max}]$ e raio do jogador r , se tem $x + r > x_{\min} \wedge x - r < x_{\max} \wedge z + r > z_{\min} \wedge z - r < z_{\max}$.

- **Interface** – WebGL para o jogo; HTML/CSS para HUDs, modais (ATM, Bar) e UI dos jogos. Cada controlador (`BlackjackController`, etc.) faz a ponte entre o estado da partida e o HUD.
- **Animacao** – duas vias coexistem: `AnimationManager` (tweens promissorios processados num `update()` central) e RAF locais para animacoes auto-contidas (roleta, slots, queda no blackout).

10 Desempenho e otimizacoes

- **Pixel ratio limitado** – `min(devicePixelRatio, 1.75)` em geral e `min(., 1.05)` em `lowPower`.
- **Sombras** – so a `DirectionalLight` principal tem mapa de sombras, e este e desactivado em `lowPower`.
- **PMREM** – gerado uma unica vez no arranque (e nao em `lowPower`).
- **Materiais e geometrias partilhados** – `ProceduralTextures` cria cada material uma vez; o mesmo material e reutilizado por dezenas de meshes.
- **Nevoeiro adaptativo** – raios `near/far` reduzidos em `lowPower` para cortar trabalho de fragmento.
- **Frustum culling** explicitamente activado em `lowPower` para todos os meshes.
- **Animacoes baseadas em Δt** – independentes do framerate efetivo.
- **Build minificada** via Vite + Terser; deploy estatico em GitHub Pages.

11 Compatibilidade

- **Desktop**: navegadores modernos com WebGL2; testado em Firefox e Chromium.
- **Mobile / tablet**: suportado em ecras `pointer: coarse` ou com `maxTouchPoints > 0`; UI de toque criada automaticamente. As animacoes sao iguais, mas o pixel ratio, sombras e ambient map sao reduzidos.
- **Publicacao**: GitHub Pages atraves de workflow CI (`.github/workflows/deploy.yml`) que executa `npm ci`, `npm run build` e publica o conteudo da pasta `dist`. URL esperado: https://t4v4res.github.io/casino_royale/.

12 Uso de IA

A IA foi usada como apoio tecnico ao longo do desenvolvimento. Todas as sugestoes foram revistas e integradas manualmente; as decisoes finais de mecanica, tuning e implementacao sao do autor.

- **ChatGPT** – foco em logica e explicacao tecnica:
 - Organizacao modular do README e do relatorio.
 - Validacao da configuracao Vite + GitHub Pages.
 - Sugestoes de iluminacao e decoracao interior.
- **Claude** – foco em arquitetura e refatoracao:
 - Pipeline de input touch e deduplicação de `click/touchend` em mobile.
 - Animacao das slots e refatoracao da trajetoria da bola da roleta.
 - Polimento de CSS/UI do HUD e modais.

13 Referencias

- [Three.js](#) – biblioteca grafica WebGL.
- [Three.js documentation](#).
- [Vite](#) – ferramenta de build.
- Penner, R. – *Easing Functions*; easings.net.
- Three.js examples – `RoomEnvironment`, `PMREMGenerator`.
- Knuth, D. – *The Art of Computer Programming, Vol. 2*: descricao de Fisher-Yates.